

UNIVERSIDAD DE MÁLAGA

Doble Grado en Ingeniería Informática y
Matemáticas

K-COLOR es NP-completo

Estudio de la reducción desde 3-SAT utilizando gadgets

Juan Elias Lara Soto

2 de mayo de 2025

Índice

1. **Introducción**
2. **Definición del problema como lenguaje**
 - 1.1) Ejemplo válido
 - 1.2) Ejemplo inválido
 - 1.3) Aplicaciones del problema
3. **Demostración de que el problema es NP-completo**
 - 2.1) El problema pertenece a NP
 - 2.2) Reducción desde un problema NP-completo
 - a) Descripción del problema fuente
 - b) Ejemplo
 - c) Campo de aplicación del problema fuente
 - d) Construcción del gadget con ejemplos
 - Ejemplo válido
 - Ejemplo inválido
 - e) Demostración de la doble implicación
 - Si φ es satisfacible, entonces el grafo es k -coloreable
 - Si el grafo es k -coloreable, entonces φ es satisfacible
 - f) Tiempo polinómico de la transformación
4. **Preguntas sobre el ejercicio resuelto**
5. **Referencias bibliográficas**

1. Introducción

Los problemas de decisión en teoría de la complejidad computacional constituyen uno de los pilares fundamentales para entender los límites de la computación eficiente. Entre ellos, destaca de forma especial la clase de problemas denominados **NP-completos**, cuya resolución en tiempo polinómico implicaría resolver eficientemente toda la clase NP.

En este trabajo nos centraremos en uno de estos problemas emblemáticos: **k-COLOR**, cuya formulación general consiste en decidir si es posible colorear los vértices de un grafo dado utilizando a lo sumo k colores, de forma que ningún par de vértices adyacentes comparta el mismo color. Esta tarea, que en apariencia puede parecer simple, encierra una complejidad computacional considerable, especialmente para valores de $k \geq 3$.

El interés por este problema no es meramente teórico. La coloración de grafos tiene múltiples aplicaciones prácticas: desde la planificación de horarios y la asignación de frecuencias en redes inalámbricas, hasta el diseño de circuitos electrónicos y la programación de tareas en sistemas operativos.

El objetivo principal de este documento es demostrar que el problema **k-COLOR** es **NP-completo para todo $k \geq 3$** , partiendo de la demostración de la NP-completitud de **3-COLOR** mediante una reducción desde el conocido problema **3-SAT**, utilizando construcciones auxiliares llamadas *gadgets*. Posteriormente, generalizaremos la demostración para valores arbitrarios de k .

Para garantizar el rigor formal, se seguirá el esquema clásico de prueba de NP-completitud: definición formal del lenguaje, demostración de pertenencia a NP, construcción de una reducción polinómica desde un problema NP-completo conocido (3-SAT) y análisis detallado de la doble implicación lógica entre ambas formulaciones.

2. Definir el problema en forma de lenguaje

Un *coloreado* de un grafo es una asignación de colores a sus nodos de forma que ningún par de nodos adyacentes tenga el mismo color.

Sea el lenguaje:

$$3\text{COLOR} = \{ \langle G \rangle \mid G \text{ es coloreable con 3 colores} \}.$$

Demuéstrese que **3COLOR** es NP-completo.

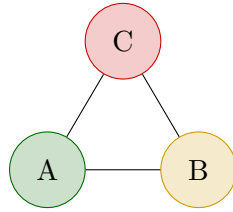
Además, dado un número natural $k \geq 3$, definimos el lenguaje generalizado:

$$k\text{-COLOR} = \left\{ \langle G \rangle \mid \begin{array}{l} G \text{ es un grafo y es posible colorear sus nodos con } k \text{ colores} \\ \text{de modo que ningún par de nodos adyacentes comparta color} \end{array} \right\}.$$

Demuéstrese que el problema $k\text{-COLOR}$ es **NP-completo** para todo $k \geq 3$.

2.1. Ejemplo válido

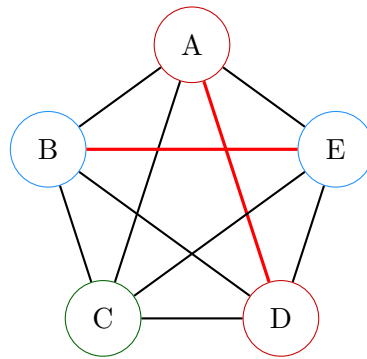
Grafo $G_1 = (\{A, B, C\}, \{(A, B), (B, C), (A, C)\})$: un triángulo completamente conectado que puede colorearse con tres colores distintos, uno por nodo.



2.2. Ejemplo inválido

Grafo $G_2 = (\{A, B, C, D, E\}, \{(A, B), (A, C), (A, D), (A, E), (B, C), (B, D), (B, E), (C, D), (C, E), (D, E)\})$

G_2 está totalmente conectado con cinco nodos. No puede colorearse con solo tres colores sin conflictos.



2.3. Aplicaciones de 3-COLOR y k -COLOR ($k > 3$)

- **Planificación de horarios y asignación de recursos:** En la programación de horarios escolares o universitarios, se puede modelar cada clase como un nodo en un grafo, y se conectan con una arista si comparten estudiantes o profesores. Una coloración adecuada asigna diferentes horarios (colores) a clases que no pueden ocurrir simultáneamente, minimizando conflictos.
- **Asignación de frecuencias en redes inalámbricas:** En telecomunicaciones, las estaciones base que están geográficamente cercanas deben operar en diferentes frecuencias para evitar interferencias. Modelando cada estación como un nodo y conectando aquellas que están dentro del rango de interferencia, una coloración del grafo asegura una asignación eficiente de frecuencias.
- **Diseño de circuitos electrónicos (VLSI):** En el diseño de circuitos integrados, componentes que no deben activarse simultáneamente se representan como nodos conectados. Una coloración adecuada ayuda a organizar las señales de control y minimizar interferencias, optimizando el rendimiento del circuito.

- **Coloración de mapas:** Aunque el Teorema de los Cuatro Colores establece que cualquier mapa plano puede colorearse con cuatro colores sin que regiones adyacentes compartan color, en mapas más complejos o en representaciones tridimensionales, puede requerirse más de cuatro colores. La coloración de grafos ayuda a determinar el número mínimo de colores necesarios para tales mapas.
- **Programación de tareas en sistemas operativos:** En la asignación de procesos a recursos compartidos, como CPUs o impresoras, se puede modelar cada proceso como un nodo y conectar aquellos que compiten por el mismo recurso. Una coloración del grafo asegura que procesos que requieren el mismo recurso no se ejecuten simultáneamente, evitando conflictos y mejorando la eficiencia del sistema.

3. Prueba de que el problema es NP-Completo

3.1. Pertenencia a la clase NP

Demostración 1: 3-COLOR $\in$ NP

Para demostrar que el problema 3-COLOR pertenece a la clase NP, basta con proporcionar una forma de certificado y un algoritmo verificador que pueda validarlo en tiempo polinómico.

- El **certificado** consiste en una asignación de uno de los tres colores posibles a cada vértice del grafo, de modo que ningún par de vértices conectados comparta el mismo color.

El **verificador** recibe como entrada un grafo $G = (V, E)$ y una asignación de colores c y debe comprobar si se trata de una 3-coloración válida. Para ello, realiza las siguientes comprobaciones:

- Recorre cada arista $e = \{u, v\} \in E$, y verifica que los colores asignados a los extremos u y v son distintos.
- Si todas las aristas conectan vértices con colores diferentes, se acepta la certificación; en caso contrario, se rechaza.

Este procedimiento puede llevarse a cabo en tiempo polinómico:

- $O(|E|)$ para recorrer las aristas del grafo.
- $O(1)$ para verificar que los colores de los dos extremos de cada arista son distintos.

Dado que la verificación completa puede realizarse en $O(n^2)$ como máximo, ya que el número de aristas como máximo en un grafo será de n^2 , concluimos que el problema cumple con las condiciones para pertenecer a NP.

$$\Rightarrow \mathbf{3-COLOR \in NP}$$

3.2. Reducción desde 3SAT

3.2.1. Descripción del problema fuente

El problema **3-SAT** (abreviatura de *3-Satisfiability*) es un problema de decisión en lógica proposicional. Se formula de la siguiente manera:

Dada una fórmula booleana φ en **forma normal conjuntiva** (CNF), donde cada cláusula contiene exactamente **tres literales**, se pregunta:

¿Existe una asignación de valores de verdad a las variables de φ que haga verdadera toda la fórmula?

Una fórmula está en forma CNF si es una conjunción (operación lógica AND) de cláusulas, y cada cláusula es una disyunción (operación lógica OR) de literales (variables o negaciones de variables).

Formalmente, una instancia de 3-SAT tiene la forma:

$$\varphi = (l_{1,1} \vee l_{1,2} \vee l_{1,3}) \wedge (l_{2,1} \vee l_{2,2} \vee l_{2,3}) \wedge \cdots \wedge (l_{m,1} \vee l_{m,2} \vee l_{m,3})$$

donde cada literal $l_{i,j}$ es una variable proposicional x_k o su negación $\neg x_k$.

3.2.2. Ejemplo

Considérese la fórmula:

$$\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4) \wedge (\neg x_3 \vee \neg x_4 \vee x_2)$$

Una posible asignación que satisface la fórmula es:

$$x_1 = \text{TRUE}, \quad x_2 = \text{TRUE}, \quad x_3 = \text{FALSE}, \quad x_4 = \text{FALSE}$$

Verificamos que con esta asignación, cada cláusula es verdadera:

- $(x_1 \vee \neg x_2 \vee x_3) = \text{TRUE} \vee \text{FALSE} \vee \text{FALSE} = \text{TRUE}$
- $(\neg x_1 \vee x_2 \vee x_4) = \text{FALSE} \vee \text{TRUE} \vee \text{FALSE} = \text{TRUE}$
- $(\neg x_3 \vee \neg x_4 \vee x_2) = \text{TRUE} \vee \text{TRUE} \vee \text{TRUE} = \text{TRUE}$

3.2.3. Campo de aplicación

El problema 3-SAT tiene aplicaciones fundamentales en muchas áreas de la informática y la inteligencia artificial. Un campo destacado es la **verificación de circuitos lógicos**:

En ingeniería de hardware y software, se necesita comprobar que un diseño lógico (como un circuito digital o un programa) cumple ciertas propiedades. Estas propiedades se pueden traducir en fórmulas booleanas. Verificar si existe una asignación que hace verdadera la fórmula equivale a comprobar si el diseño puede alcanzar un estado indeseado o si cumple una especificación.

También se aplica en:

- Planificación automática en robótica e IA

- Resolución de problemas en lógica formal y juegos
- Teoría de bases de datos y restricciones

3-SAT es el prototipo de problema **NP-completo**, lo que significa que si se encontrara un algoritmo eficiente para resolverlo, se podría resolver eficientemente cualquier problema en la clase NP.

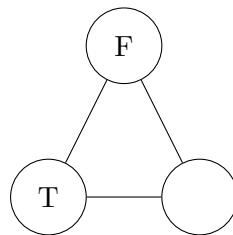
3.2.4. Construcción del gadget con ejemplos

Primero veremos que podemos reducir en tiempo polinómico el problema *3SAT* a *3-COLOR*, luego 3-COLOR es NP-Completo, y después daremos una generalización a *k-COLOR* para cualquier $k > 3$.

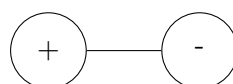
Explicación del gadget

1. Gadget. Comenzamos construyendo el gadget, el cual mediante una fórmula booleana en 3-CNF nos genere un grafo coloreable. Usamos la pista que nos proporciona el enunciado del ejercicio 38 del tema 7 del Spser. Se nos proponen los siguientes grafos:

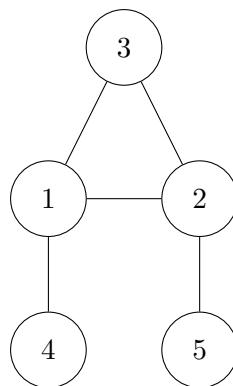
- **Palette gadget:** tres nodos conectados (T, F, R) que fijan los tres colores.



- **Variable gadget:** dos nodos (+, -) conectados a R.



- **OR gadget:** simula disyunciones, con conexiones según la aparición del literal.



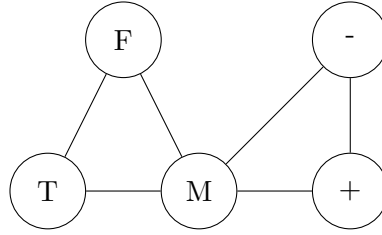
Luego, sea $\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_l$, una fórmula con l cláusulas sobre el conjunto de variables booleanas $\{x_1, \dots, x_m\}$.

Construimos un grafo G_φ (gadget) de la siguiente forma:

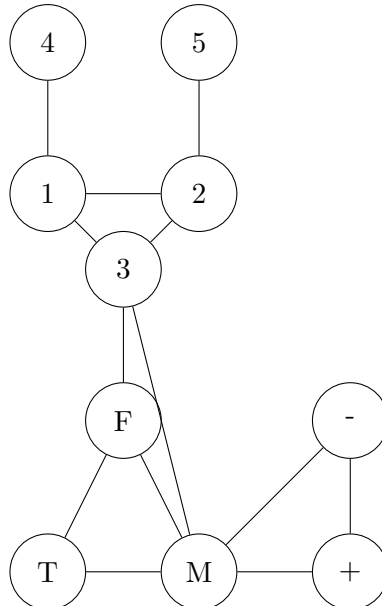
- Usamos **dos variable-gadgets por cada variable** booleana x_i .
- Usamos **dos OR-gadgets por cada cláusula** C_j .
- Usamos **un único palette-gadget**.

ELABORACIÓN DE LA REDUCCIÓN

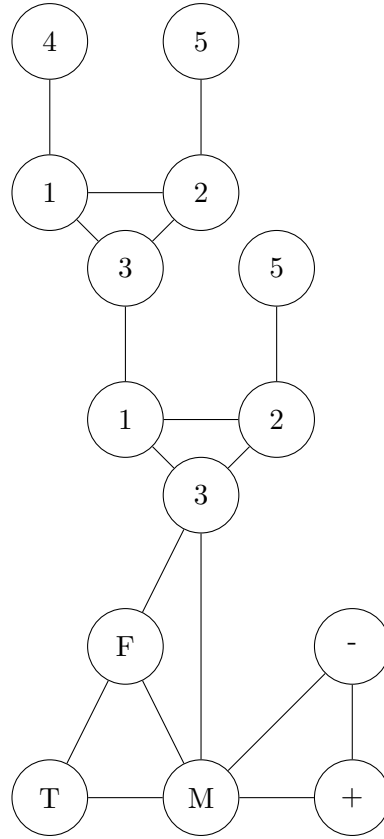
1. A cada variable lógica le asociamos un **variable-gadget**, el cual etiquetamos con los símbolos $+$ (variable true) y $-$ (variable false). Este gadget se conecta directamente con el nodo M del palette, cuyos nombre de los nodos cobrarán sentido a posteriori (En este ejemplo, se representa únicamente un gadget de variable como si hubiera una sola variable en la fórmula.).



2. 1 A cada cláusula de la fórmula le corresponden dos **OR-gadgets**. En este paso, conectamos el nodo superior, esto es, el nodo 3 del primer OR-gadget, con el nodo F del palette definido previamente. Además, dicho nodo 3 también se conecta con el nodo M, cerrando así la parte superior de la estructura.

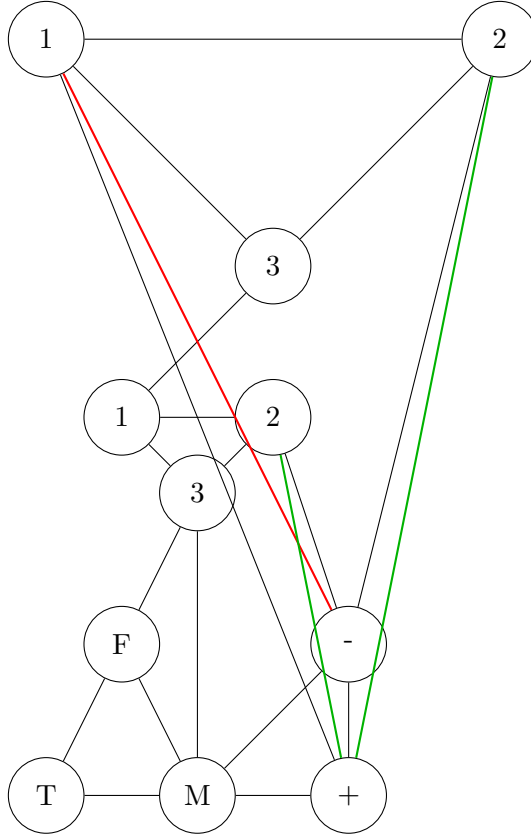


2.2 A continuación, reemplazamos uno de los nodos inferiores del primer OR-gadget por el nodo superior del segundo OR-gadget. Así se encadena el nuevo gadget a la estructura. En este proceso, el nodo 4 es eliminado y sustituido por el nuevo subgrafo.



3. Ahora conectamos los nodos 4,5 y 5 con con el **variable-gadget** correspondiente. Si la cláusula incluye la variable x_i , se hace corresponder con el nodo + del variable-gadget asociado. Si en su lugar aparece $\neg x_i$, se vincula con el nodo -. Por ello el nodo 4 pasa a ser el mismo que - y el nodo 5 pasa a ser el mismo que mas.

En este dibujo, se ha representado una sola variable, y por tanto, un solo variable-gadget. Sin pérdida de generalidad, las aristas rojas y verdes podrían ser un ejemplo valido para el gadget correspondiente para una 3-CNF dada.



4. Asignamos colores a los nodos del grafo según el *palette-gadget*: T (verde), F (rojo) y M (marrón).

Finalmente, debemos abordar el coloreado de los nodos en el grafo. Para ello, utilizamos las referencias proporcionadas por el *Palette-Gadget*, en el que los símbolos T , F y M representan colores distintos.

Para cada variable, asignamos al nodo $+$ el color T y al nodo $-$ el color F en el *Variable-Palette*, siempre que dicha variable tome el valor verdadero en una asignación que satisfaga φ (lo cual sucederá si φ es satisfacible). En caso contrario (si φ es insatisfacible, por ejemplo), invertimos esta asignación de colores.

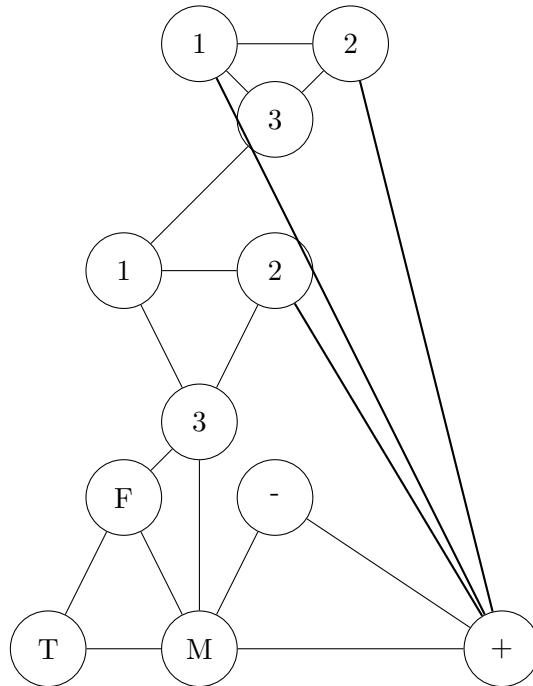
Como cada cláusula posee al menos un literal que se evalúa como verdadero bajo una asignación satisfactoria, existirá al menos una entrada al *OR-Gadget* que esté coloreada con T . Esto permite colorear adecuadamente los nodos del *OR-Gadget* de forma que el nodo conectado al F del *Palette-Gadget* no adopte el color rojo (F). En consecuencia, se obtiene una 3-coloración válida.

Por otro lado, si φ no es satisfacible, todas las entradas al *OR-Gadget* estarán coloreadas con F , lo que forzará a que el nodo vinculado al F del *Palette-Gadget* adopte también el color F , lo que produce una coloración no válida.

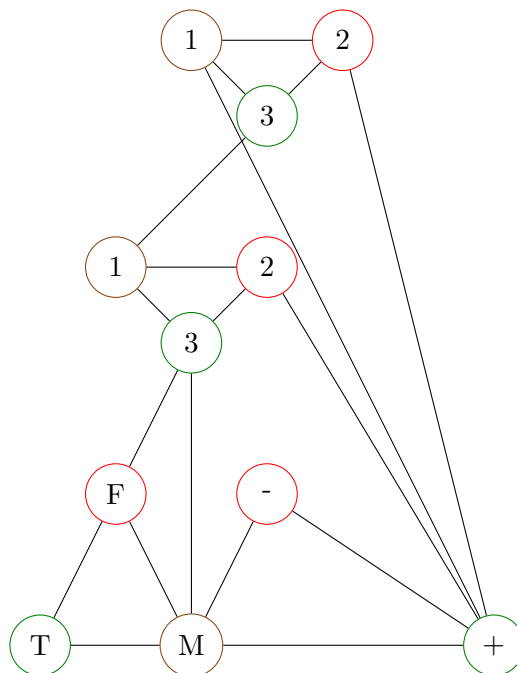
Ejemplo válido

Consideremos el ejemplo más sencillo para poder ver con claridad las aristas del grafo y que no resulte imposible distinguirlas, este es, la fórmula booleana $\varphi = (x \vee x \vee x)$, que se encuentra en 3CNF.

La fórmula φ resulta ser satisfacible, ya que, asignando a x el valor verdadero, la cláusula queda satisfecha. Dado que únicamente consta de una cláusula y una variable, el proceso de construcción del grafo G_φ se lleva a cabo exactamente como se explicó anteriormente.



Tenemos que φ es una fórmula satisfacible, luego asignamos el color del nodo T a + y el color de F al nodo -. Luego, el nodo 3 que conecta con el nodo F ha de ser verde para que sea válida la coloración. Teniendo en cuenta lo mencionado, coloreamos el grafo con una coloración válida.



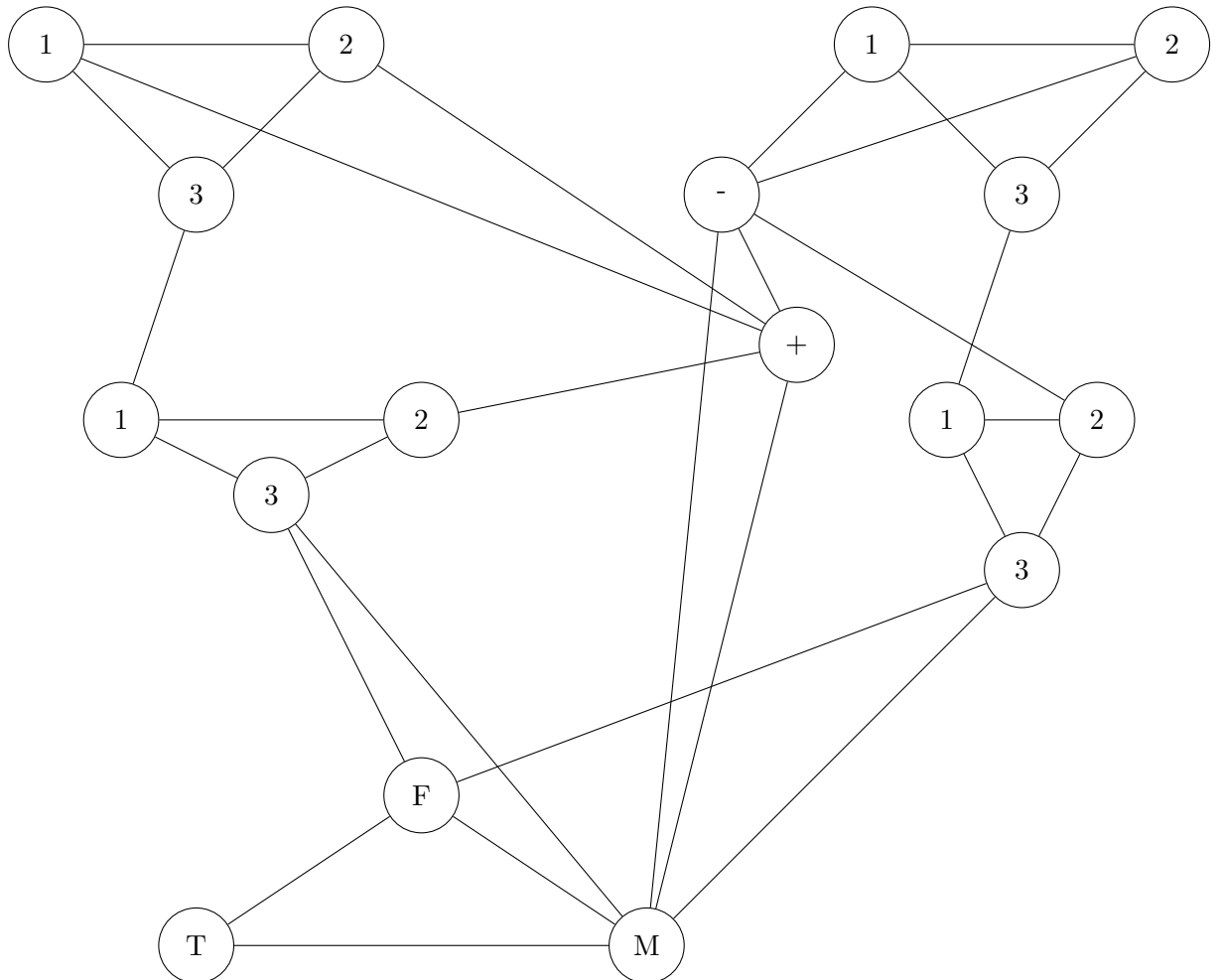
Se observa claramente, que es una 3-coloración válida, la cual se ha obtenido a partir de una fórmula satisfacible.

Ejemplo inválido

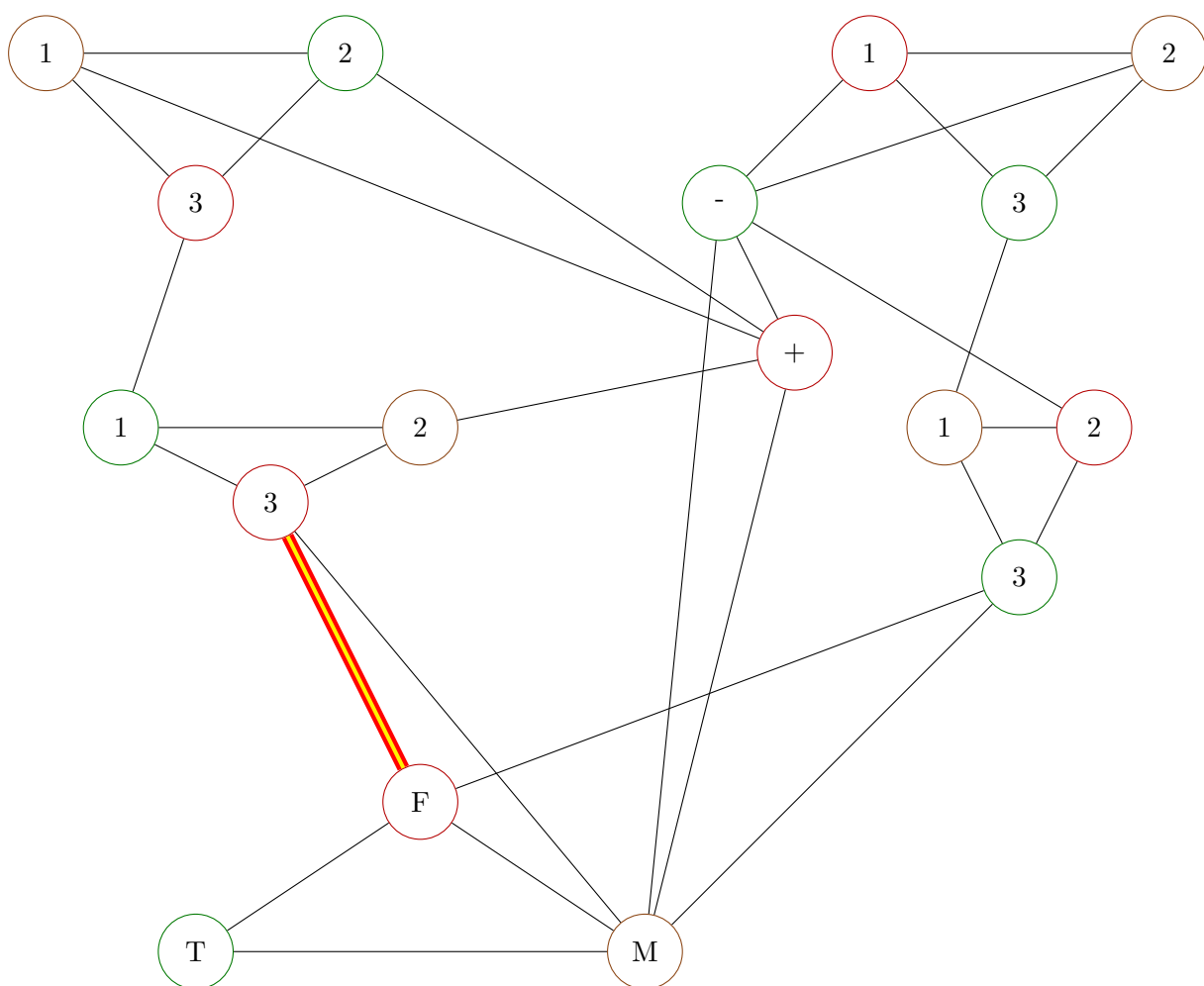
Consideremos ahora la formula 3-CNF insatisfacible más sencilla que se nos puede ocurrir: $\varphi = (x_1 \vee x_1 \vee x_1) \wedge (\neg x_1 \vee \neg x_1 \vee \neg x_1)$. Esta fórmula posee una variable y dos cláusulas.

La fórmula φ es insatisfacible, ya que, para cualquier asignación de valor de verdad a x_1 , se requiere simultáneamente que x_1 y su negación sean verdaderos, lo cual es imposible. Por ejemplo, asignar el valor 1 a x_1 no logra satisfacer φ .

Dado que el número de cláusulas es dos y sólo se maneja una variable, el procedimiento de construcción del grafo G_φ sigue la metodología descrita anteriormente, teniendo en cuenta que ahora se incorporarán dos gadgets tipo OR adicionales.



Coloreamos el palette gadget y el variable gadget teniendo en cuenta que x_1 igual a true hace falsa la fórmula booleana, luego - será verde y + será rojo. A partir de aquí coloreamos cada cláusula de forma independiente ya que cada cláusula por separado sí que es satisfacible, luego, obtenemos la siguiente 3-coloración.



Observamos que, al realizar la coloración propuesta, el nodo F del *palette gadget* y el nodo 3 del *OR-gadget central* (asociado a la segunda cláusula) están unidos por una arista y, sin embargo, han sido coloreados con el mismo color (rojo). Esto viola la condición fundamental de una 3-coloración válida, que requiere que todo par de nodos adyacentes tengan colores distintos. Por tanto, esta coloración no es válida y refleja correctamente la insatisfacibilidad de la fórmula original.

3.2.5. Demostración de la doble implicación

Teorema: 3-COLOURING es NP-completo

Demostración:

1. 3-COLOURING pertenece a NP.

Ya lo hemos visto en este documento.

2. 3-COLOURING es NP-hard.

Para probar la NP-dificultad, se da una reducción polinomial desde 3-SAT a 3-COLOURING. Es decir, dada una instancia φ de 3-SAT, veremos que el gadget construido anteriormente 3-colorable si y solo si φ es satisfacible.

Captura de cláusulas mediante gadgets OR.

Para reflejar las cláusulas de φ , se introduce un gadget OR para cada cláusula $C_i = (a \vee b \vee c)$.

Este gadget conecta los literales a, b, c y tiene un nodo de salida que representa $a \vee b \vee c$. El gadget se construye en dos etapas: primero se calcula $a \vee b$, luego $(a \vee b) \vee c$.

El gadget satisface las siguientes propiedades:

1. Si a, b y c están coloreados con F , entonces el nodo de salida debe estar coloreado también con F , reflejando que la cláusula no está satisfecha.
2. Si al menos uno de a, b o c está coloreado con T , entonces existe una 3-coloración válida del gadget en la que el nodo de salida se colorea con T , reflejando que la cláusula sí está satisfecha.

Una vez contruidos todos los gadgets OR, cada nodo de salida se conecta a los vértices F y B del triángulo inicial.

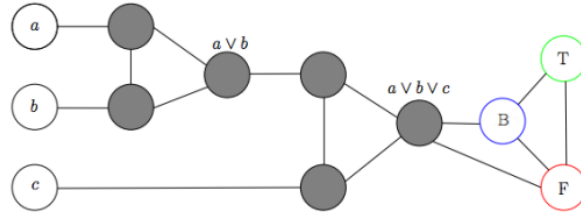


Figura 1: Mouatadid 2014

Finalización de la prueba.

($\Rightarrow$) Supongamos que φ es satisfacible, y sea $(x_1^*, x_2^*, \dots, x_n^*)$ una asignación satisfactoria. Coloreamos cada v_i con T si $x_i^* = 1$ y $\bar{v}_i$ con F ; o viceversa si $x_i^* = 0$. Dado que cada cláusula $C_i = (a \vee b \vee c)$ tiene al menos un literal verdadero, el correspondiente gadget OR puede ser 3-coloreado de modo que el nodo de salida esté coloreado T , respetando la conexión a F y B .

($\Leftarrow$) Supongamos ahora que G es 3-colorable. Definimos una asignación de valores de verdad de la siguiente forma: asignamos $x_i = 1$ si v_i tiene color T , y $x_i = 0$ si $\bar{v}_i$ tiene color T . Si esta asignación no satisficiera φ , existiría una cláusula C_i donde todos los literales fueran falsos, lo que implicaría que el nodo de salida del gadget OR estaría coloreado F , en conflicto con su conexión al vértice F del triángulo, violando así la 3-colorabilidad.

Por tanto, φ es satisfacible si y solo si G es 3-colorable.

3.2.6. Tiempo polinómico

Vamos a justificar que tanto la construcción del grafo G_φ como su proceso de 3-coloración se realizan en tiempo polinómico con respecto al tamaño de la entrada, es decir, el número de variables n y de cláusulas l .

Construcción del grafo G_φ

La generación del grafo comienza con la creación de un único *palette gadget*, que consta de un número fijo de nodos y aristas, por lo que su construcción requiere tiempo constante.

A continuación, para cada variable booleana de la fórmula, se incorpora un *variable gadget*, el cual está compuesto por dos nodos conectados entre sí. Dado que el número de variables es n , el número total de nodos y aristas añadidos por este paso es proporcional a n .

Respecto a las cláusulas, cada una se representa mediante dos *OR gadgets* modificados. Aunque la versión clásica contiene 8 nodos, en este caso se simplifican eliminando los dos nodos auxiliares finales, lo que deja 6 nodos y 7 aristas por cláusula. Como hay l cláusulas, el total de nodos y aristas introducidos por este paso también es lineal respecto a l .

En resumen, el número total de operaciones necesarias para construir todos los gadgets del grafo es proporcional a $n+l$, lo cual implica una complejidad de construcción en tiempo $\mathcal{O}(n+l)$.

Coloración en tres colores

Una vez construido el grafo, se procede a su coloración utilizando tres colores. Este proceso consiste en recorrer todos los nodos y asignarles un color de acuerdo con las reglas establecidas por los gadgets construidos. Dado que el número de nodos y aristas es lineal en función de n y l , y que cada operación de asignación de color puede realizarse en tiempo constante, el tiempo total de este proceso también es polinómico.

Conclusión

Tanto la generación del grafo G_φ como su coloración bajo la restricción de 3 colores se realizan en tiempo polinómico respecto al tamaño de la fórmula de entrada. Por lo tanto, el método es computacionalmente eficiente en términos de complejidad temporal.

3.2.7. Demostración de que k -COLOR es NP-completo para $k \geq 3$

Consideramos el problema de decidir si un grafo puede colorearse utilizando a lo sumo k colores, de modo que vértices adyacentes reciban colores distintos. Este problema, denominado k -COLOR, generaliza el conocido 3-COLOR.

Es intuitivo pensar que si un grafo es 3-coloreable, también lo será para cualquier número mayor de colores, ya que bastará con emplear únicamente tres de los k disponibles e ignorar los restantes. En este sentido, 3-COLOR puede verse como un caso particular de k -COLOR.

Vamos a demostrar formalmente que k -COLOR es NP-completo para cualquier $k \geq 3$.

Pertenencia a NP. La verificación de una k -coloración consiste en comprobar que cada par de vértices adyacentes recibe colores distintos y que el total de colores

utilizados no supera k . Usando un certificado análogo al que se utilizó para 3-color, existe un certificado con una asignación de colores a los vértices, esta comprobación puede realizarse en tiempo polinómico. Por tanto, k -COLOR pertenece a NP.

Reducción desde 3-COLOR. Para demostrar la NP-completitud, basta construir una reducción polinómica desde 3-COLOR, que ya sabemos que es NP-completo.

Dada una instancia G del problema 3-COLOR, construiremos un grafo H tal que G es 3-coloreable si y solo si H es k -coloreable.

El grafo H se obtiene de G añadiendo un subgrafo completamente conexo (clique) con $k - 3$ nuevos vértices. A continuación, conectamos cada uno de estos nuevos vértices con todos los vértices del grafo G original.

Correctitud de la reducción.

- Si G admite una 3-coloración válida, podemos extender esta coloración a H asignando a los $k - 3$ nuevos vértices colores distintos y distintos de los ya usados en G , lo cual es posible porque estos nuevos vértices forman una clique y están conectados a todo G .
- Si G no es 3-coloreable, entonces para colorear H necesitamos al menos $k - 3$ colores sólo para los nuevos vértices, y no podremos asignar colores a G usando únicamente los 3 colores restantes, ya que por hipótesis G no es 3-coloreable. Por tanto, H no será k -coloreable.

Este procedimiento de construcción es claramente polinómico en el tamaño de G , ya que sólo añade un número constante de vértices y aristas en función de k .

Conclusión. Hemos demostrado que k -COLOR pertenece a NP y que existe una reducción polinómica desde 3-COLOR. Por tanto, se concluye que k -COLOR es NP-completo para todo $k \geq 3$.

4. Preguntas tipo test sobre la reducción de 3-SAT a 3-COLOR

Pregunta 1

¿Cómo se representa una variable negada ($\neg x$) en el grafo construido a partir de una fórmula 3-CNF?

- A) Conectando el nodo $+$ del gadget de variable al OR-gadget
- B) Asignándole el mismo color que el nodo M
- C) Conectando el nodo $-$ del gadget de variable al OR-gadget
- D) Usando un gadget de cláusula adicional

Pregunta 2

¿Cómo se asegura la correspondencia entre la satisfacibilidad de una fórmula y la 3-colorabilidad del grafo construido?

- A) Cada cláusula se convierte en un triángulo coloreable
- B) El gadget OR permite que al menos una entrada tenga color T si la cláusula es verdadera
- C) Los colores se asignan al azar para simular todas las asignaciones
- D) Si el grafo tiene ciclos, entonces la fórmula no es satisfacible

Pregunta 3

¿Qué ocurre si una fórmula booleana es insatisfacible al aplicar la reducción a 3-COLOR?

- A) El grafo resultante no tiene aristas
- B) La construcción del gadget falla
- C) No existe una 3-coloración válida para el grafo
- D) El grafo se vuelve completamente conexo

5. Referencias bibliográficas

Youssef Mouatadid. *Gadget Reductions: 3-SAT to 3-COLOR*. Lecture notes, CSC373, University of Toronto, 2016.

<https://www.cs.toronto.edu/~lalla/373s16/notes/3col.pdf>

Kristin Gardner. *Reducing 3-SAT to 3-Coloring*. Course handout, COSC-311, Amherst College, 2017.

https://kgardner.people.amherst.edu/courses/f17/cosc311/handouts/graph_coloring_solutions.pdf